

PRACTICAL_11

1. Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

CODE:

```
GNU nano 7.2 thread_counter.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for(int i = 0; i < 100000; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value = %d\n", counter);

    return 0;
}
```

RESULT:

```
Hello mmap!root@SRIVALLI:~#
root@SRIVALLI:~# nano thread_counter.c
root@SRIVALLI:~# gcc thread_counter.c -o thread_counter -lpthread
root@SRIVALLI:~# ./thread_counter
Final Counter Value = 134370
root@SRIVALLI:~#
```

REPORT:

In this experiment, a multithreaded counter application was developed using POSIX threads. Multiple threads updated a shared counter variable concurrently using pthread_create() and pthread_join(). Since the shared variable was accessed without synchronization, race conditions occurred, resulting in inconsistent counter values. This experiment helps in understanding thread creation, thread synchronization issues, and race conditions in multithreaded programming.

2. Modify the previous program using mutex locks (pthread mutex lock, pthread mutex unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization

CODE:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for(int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano mutex_counter.c
root@SRIVALLI:~# gcc mutex_counter.c -o mutex_counter -lpthread
root@SRIVALLI:~# ./mutex_counter
Final Counter Value = 200000
```

REPORT:

In this experiment, mutex locks were used to synchronize access to a shared counter variable. The mutex prevented race conditions by allowing only one thread to update the counter at a time. As a result, the program produced correct and consistent output, demonstrating the importance of synchronization in multithreaded programming.